

COMP101
Introduction to Programming
2025-26
Week-02 Lab-02
Not supervised

Objectives:

Exercise-01: IDLE environment and Notepad++

Exercise-02: Output control; print() function; Escape characters; House Style

Exercise-03: Quote problems – Reading only

Exercise-04: i/o control

Exercise-05: i/o control

Exercise-06: String methods

Exercise-07: Subscripting

Exercise-08: Concatenation

Exercise-09: Type identification

Exercise-10: Getting help in the shell

Reading Only Section:

Zen of Python, Coding Style (PEP 8), Colour Defaults

Exercise-01: IDLE Environment using Notepad++

See Week-01 Lab-01 so you know how to access IDLE.

We can type code directly into IDLE ('Edit Window') as done in Workbook-01.

However, there is an alternative - try the following:

1.1

Use Notepad(++) and type in the code for 'Hello World'

```
print ('Hello World')
```

Save it as a Notepad file (not as a .py file)

1.2

Load the Notepad file into the IDLE editor and execute it

Note:

It is easier and better to just use IDLE to code your programs, although this Notepad(++) approach may come in handy.

Exercise-02: Output control

We have seen the `print()` function has to meet certain criteria to work properly:

- MUST have two balanced parentheses in the function
- Strings are enclosed in single **or** double quotes
 - `print ('Hello world')` works the same as
 - `print ("Hello world")`
- Don't mix different quotes in the same parentheses
- Keep to the same style (single or double) throughout the code
- Everything inside the quotes will output as typed
 - if there are 4 spaces between the two words, they will output with 4 spaces between them on screen
- Output from `print ()` is left justified to the screen
- Use double quotes for the 'house style'

2.1 Escape characters

We can take control over where/how on screen we want output to appear using ‘escape characters’:

- `\t` for a single tab
- `\n` for a newline
- `\”` for a quote
- `\\` for a backslash
- `end=“ ”` will output several `print()` statements onto one output line on one line
- `#` for comments (output in red)
- `print ()` to print a blank line
- `\` To split a long line if it is too long to ‘fit’ on one line of the editor
- `/` forward slash does not need to be escaped

Try the following – study the output to see what’s going on:

(i)

```
print (“Hello World”)
Hello World
```

(ii)

```
print (“\tHello World”)
    Hello World
```

(iii)

```
print (“Hello\nWorld”)
Hello
World
```

(iv)

```
print (“Hello\n    World”)
Hello
    World
```

(v)

```
print (“Hello \”World\””)
Hello “World”
```

(vi)

```
print (“Hello \\World”)
Hello \World
```

(vii)

```
print (“Hello”)
print (“World”)
Hello
World
```

(viii) `end=""`

#suppresses newline char two lines of code o/p as a single line

```
print ("Hello",end="")
```

```
print ("World")
```

Hello World

(ix) `print()`

print a blank line – several ways to do it

```
print()
```

```
print("")
```

```
print('')
```

(x) long statements

Long statements can exceed the width of the editor window.

Use the escape character \ at the end of the first line and quotes to split it up in the editor window.

Although the statement takes up two lines in the editor it outputs as a single line on screen.

Note where the quotes go

```
print ("This is a very very very very very very \
```

```
"very long line")
```

This is a very very very very very very long line

Exercise-03: The problem with quotes – reading only

In the `print()` function, outputting quotes and apostrophes within a string may cause a problem.

In Liverpool, there was a famous comedian called Sir Ken Dodd. His father owned a dog, and when the dog died a newspaper printed the following:

Ken Dodd's Dads' Dog's Dead!

If we want to output this string with the single quotes as shown, we can consider the use of swapping the type of quote (single, double or triple) or use escape characters (sequences).

Using escape sequences can make the code difficult to read if there is a lot of punctuation to deal with, so an approach using alternative quotes may be better.

In the above, the solution is to use double quotes in the `print()` function.

(i) Using single quotes

```
print ('Ken Dodd's Dad's Dog's Dead!')
```

Will give an error.

Single quotes that surround the string are clashing with the single quotes within the string. The interpreter ‘thinks’ that each time it finds a single quote, it is either closing or opening a new string, so the ‘open/close’ balance is affected and the interpreter ‘throws’ an error.

(ii) Using double quotes:

```
print ("Ken Dodd's Dad's Dog's Dead!")
```

This works because if we study the string, it only contains single quotes which won’t clash with the double quotes

The opposite would also be true: If the string only contained double quotes, we can surround the string with single quotes that won’t clash.

3.1 Strings with mixed quotes in them

There is a problem when a string contains single and double quotes as in the following example:

Doesn't the broomstick look like the "Nimbus 2000" destroyed by the "Whomping Willow" in "Harry Potter and the Goblet of Fire". One of J.K. Rowling's books?

(i) Use escape sequences within the string

Using double quotes (") to enclose the string and apply six escape sequences to handle the double quotes in the string.

The single quotes are okay – they don't clash with the double quotes that enclose the string.

#problem here is readability – lot of escape sequences and putting them in the right places

```
print ("Doesn't the broomstick look like the \"Nimbus 2000\"  
destroyed by the \"Whomping Willow\" in \"Harry Potter and  
the Goblet of Fire\". One of J.K Rowling's books?")
```

(ii) Use single quotes to surround the string

#double quotes in the string don't need escaping – they don't clash with the single quotes and we only need two escape sequences instead of six as before.

```
print ('Doesn\'t the broomstick look like the "Nimbus 2000"  
destroyed by the "Whomping Willow" in Harry Potter and the  
Goblet of Fire. One of J.K. Rowlings\'s Books?')
```

(iii) Use triple quotes

Neither single nor double quotes in the string clash with the triple quotes and we don't need any escape sequences.

```
print ("Doesn't the broomstick look like the "Nimbus 2000  
destroyed by the "Whomping Willow" in Harry Potter and the  
Goblet of Fire. One of J.K. Rowlings's Books?")
```

3.2 Strings with slashes in them

You want to output the following message:

Create a new directory called "C:\\names"

Study the problem:

- Look at the first '\\' character.
This clashes with what we know – that the '\\' character is used for an escape sequence.
- Look at the second '\\' character.
In the \\names part of the string, the first '\\' is treated as an escape sequence and the second '\\' is followed by 'n' so it is reads as '\\n' which is the newline sequence – the interpreter reads this to output a new line on screen.
- Look at the double quotes in the output string:
These will clash with the double quotes (if used) in the print statement.

Study the following scenarios:

(i)

Not an acceptable solution – it changes the specification which is to output two ‘\’ symbols in front of the word ‘names’

Also introduces a new problem with \n because \n within the nested quotes is the escape character for newline

```
print (‘ Create a new directory called “C:\names” ’)
```

```
Create new directory called "C:  
names"
```

The single quotes at start and end resolve the double-quotes clash within the output string “C:\names” but \n creates a new output line as shown

(ii)

This attempts to meet the problem specification ... but

The first ‘\’ is treated as an escape character – so it escapes the second ‘\’

the second ‘\’ now outputs as a normal character – and doesn’t escape the ‘n’ # so we don’t get a newline produced with \n

```
print (‘Create a new directory called ” C:\\names” ’)
```

```
Create new directory called "C:\names"
```

Only one backslash

(iii)

Still attempts to meet the problem specification ... but

the first ‘\’ escapes the second ‘\’ so we output a single ‘\’

the third ‘\’ escapes character n so we get a newline

```
print (‘Create a new directory called ” C:\\\\names “’)
```

```
Create new directory called "C:  
names"
```

Only one backslash – and a
newline produced

(iv)

SOLUTION: We need four ‘\’ characters

The first ‘\’ escapes the second ‘\’ so we output a single ‘\’.

#The third ‘\’ escapes the fourth ‘\’ so we output another single ‘\’

print (‘Create a new directory called ” C:\\\\names “’)

Create new directory called "C:\\names"

This works

(v)

#Alternative solution

the use of single quotes around the entire string does not

#conflict with the double quotes within the substring

print (‘ Create a new directory called ” C:\\names “ ’)

Create new directory called "C:\\names"

This works also

(vi)

#Another alternative solution – use triple quotes

#triple quotes around the entire string does not conflict with the double quotes inside it or if single quotes are present in the string also

print (‘’’ Create a new directory called ” C:\\names “ ’’’)

Create new directory called "C:\\names"

This works
as well

Exercise-04: I/O Control

(i)

Prompt the user to enter their first (given) name, (e.g. user enters Yuan)

(ii)

Output a message “Welcome to Yuan’s World!”

You need to assign user input to a variable and then use that variable in the output string in between the words ‘to’ and ‘World!’ Use the comma to control your output spacing.

Exercise-05: I/O Control

(i) If you attempted it, retrieve the menu program from Week-01 Lab-01 and go to (ii) below. Else code a menu as shown (the box around it isn’t needed)

Main Menu	

D:	Division
M:	Multiplication
A:	Addition
S:	Subtraction

(ii) Extend the menu to provide some user interaction: If you haven’t already done it last week, add a prompt to ask the user to enter an option. Put a blank line between this and the menu

(iii) Add a confirmation message to output the option the user selected.

Put a blank line between this and the prompt

NOTE: No validation etc is required at this stage

Exercise-06: String Methods

(i) Prompt the user to enter two names, lower case and separated by a space.

Try outputting with print methods for upper, lower and title

Exercise-07: Subscripting

(i) Prompt the user to enter the word ‘mathematicians’
Output (using subscripts) the substring ‘thematic’

Exercise-08: Concatenation

(i) Given the following code fragment:

```
age = int(input("Enter your age: "))  
decade = int(10)  
age_in_ten = (age+decade)
```

Extend the code to output the following message (assuming user inputs 21 as their age)

You are 21 and in ten years you will be $21+10=31$

Output can be done more easily using the comma separator - but try it using only concatenation – no commas.

Remember – concatenation only works on strings.

Exercise-09: Command shell - type identification

Can be useful when your program does not work, e.g. maybe you are using the wrong data type etc

The 'type' command identifies the type of data being used

(i) In the command shell, try the following:

```
>>> pi = 3.142
```

```
>>> type (pi)
```

See what you get (should be 'float')

(ii) In the command shell, try the following at the >>> prompt (there are 3 commands in total)

a) >>> `city_in_England = input("West coast: ")`

When you press return, you should see the prompt 'West coast: '

Execution halts until you enter a value. Type in Liverpool.

West coast: Liverpool

b) Then try the following to output the value

```
>>> print(city_in_England)
```

You should get output of Liverpool

c) Then try the following to see the data type of the variable 'city_in_England

```
>>> type(city_in_England)
```

You should get 'str'

The shell commands can be useful for checking code – although you can see that many lines of code are best done in IDLE – imagine if this was a long program . . .

Exercise-10: Command shell – help

(i) At the interactive prompt in the shell window, type:

```
>>> help()
```

You should see the help prompt

ii) At the help prompt type:

```
help> keywords
```

You should get a list of reserved words ('keywords').

You can't use these as variable names – their use is restricted by Python for special purposes and will produce an error in your program.

If you look for help on any individual keyword, the messaging can be a bit confusing as it is intended for developers - don't study it too much.

iii) Exit help

```
help> quit
```

The Zen of Python – for reading only

Python is designed to keep things easy and simple. This ‘philosophy’ is supported by ‘The Zen of Python’ to encourage clear and simple thinking in problem solving.

Take a look at:

<https://www.python.org/dev/peps/pep-0020/>

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Flat is better than nested.
- Sparse is better than dense.
- Readability counts.
- Special cases aren't special enough to break the rules.
- Although practicality beats purity.
- Errors should never pass silently.
- Unless explicitly silenced.
- In the face of ambiguity, refuse the temptation to guess.
- There should be one-- and preferably only one --obvious way to do it.
- Although that way may not be obvious at first unless you're Dutch.
- Now is better than never.
- Although never is often better than **right** now.
- If the implementation is hard to explain, it's a bad idea.
- If the implementation is easy to explain, it may be a good idea.

Python Coding Style – reading only

As your code gets bigger, a coding style ('House Style') should be adopted to make it easier to read and maintain. The main 'style guide' for Python is PEP 8 – don't memorise it! It is okay in your code if it does not follow the following advice exactly:

Try the following:

i)

Take a look at <https://www.python.org/dev/peps/pep-0008/>

ii)

Browse through the guide – guidance notes are below:

#id17, #id18 – indentation and spaces

use 4-space indentation, no tabs

#id19 – line length

Line length max = 79 chars

#id20 – line breaks with operators

Optional

#id21 – blank lines

Blank lines should separate functions

#id22

Ignore

#id23 - imports

Use imports on separate lines

#id24

Ignore

#id25 – string quotes

Use double quotes (triple quotes in exception)

#id26, #id27, #id28 - whitespace

Optional (keep it tidy for clarity and readability)

It is usual to use spaces around operators and after commas, but not usually inside brackets

#id29, #id30(ignore), #id31 - comments

Put comments above the code they refer to

Ignore other detail under #id29

Use inline comments sparingly

#id32

Ignore

#id33, #id34, #id35 – naming conventions

For any names you have to make up yourself
(functions and variables) use lowerCamelCase or
use mixedCase

Ignore other detail under #id33, 34, 35

#id36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47 48, 49 50

Ignore all of these

Python code - colour defaults **Reading only**

In IDLE, Python colours the code depending on what the code is being used for: The default colours are:

Function names from library **Purple**

Function name (user-defined) **Blue**

Strings **Green**

Names, Numbers, Symbols **Black**

Output **Blue**

Keywords **Orange**

Comments and errors **Red**

Don't learn these, but become familiar with them